



BILDUNG. FREUDE INKLUSIVE.

Webdesign im IT-L@B

Einführung in HTML und CSS – Modul 15

Grid-Layouts mit CSS

Überarbeitet: 01.10.2025

von Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

LESEN (!!!!) Sie dieses Skript aufmerksam durch!!!

Arbeitsauftrag:

1. **Lesen** Sie das Skript **vollständig**!
2. **Programmieren** Sie die vorgestellten Beispiele – auch in eigenen Varianten – nach. Vermeiden Sie es, fremde Programmskripte zu kopieren!
3. **Bearbeiten** Sie den am Ende der Lektion gestellten Arbeitsauftrag!
4. **Testen** Sie die Webseite auf verschiedenen Webbrowsern!

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen!!!!

Kalkulierte Bearbeitungszeit

Die Aufgabe ist so gestaltet, dass Sie einschließlich der Lektüre dieser Lektion, dem Ausprobieren der Beispiele und eventuell eigene Recherchen im Internet maximal **16 Stunden Arbeitszeit** benötigen sollten. Wenn Sie Probleme (z.B. Schwierigkeiten mit dem Verständnis der deutschen Sprache) haben sollten, wenden Sie sich bitte an Ihre Trainerin oder Ihren Trainer.

Für die Abgabe der Aufgaben gelten folgende Regeln:

Offizielle Lehrlinge des IT-L@B:

Lehrlinge des IT-@B verfügen über einen Zugang zum Lernsystem Moodle. Rufen Sie den folgenden Kurs auf:

20E 55 - User Experience - IT-L@B Villach

Geben Sie die Arbeit im dafür vorgesehenen Abgabefeld der Aufgabe *fristgerecht* ab.

Praktikantinnen und Praktikanten:

Praktikantinnen und Praktikanten haben noch keinen Zugang zum Moodle-System. Sie geben die Aufgabe bitte in Ihrem **persönlichen Ordner** auf dem NAS („\\itlabvi\\public“) ab. Richten Sie dafür bitte ein Unterverzeichnis „web“ ein. Das Speichern auf einem anderen Verzeichnis – z.B.: auf einem lokalen PC – gilt als NICHT abgegeben!

Inhalt

LESEN (!!!!) Sie dieses Skript aufmerksam durch!!!	2
Arbeitsauftrag:	2
Kalkulierte Bearbeitungszeit	2
Für die Abgabe der Aufgaben gelten folgende Regeln:	2
Offizielle Lehrlinge des IT-L@B:	2
Praktikantinnen und Praktikanten:	2
Die CSS-Eigenschaft Grid	4
Der Grid-Container	4
Struktur-Deklaration	7
grid-template-rows	7
grid-template-columns	8
grid-template	10
Einordnung der Grid-Item in das Raster	10
grid-column	10
grid-row	12
Abstände im Raster: gap, row-gap und column-gap	14
grid-area	15
grid-template-area	15
grid-area ohne Namen	17
Ausrichtung der Elemente	17
Übungsaufgaben	18

Die CSS-Eigenschaft Grid

Thema des vorherigen Moduls 14 war der Einsatz der CSS-Flex-Technologie u.a. zur mehrspaltigen und responsiven Gestaltung einer Webseite. Beim Flex-Layout haben wir es mit einem „*eindimensionalen*“ Design zu tun, welches entweder an Zeilen oder an Spalten ausgerichtet wird und an den Grenzbereichen – sofern es gewünscht ist – *Umbrüche* erzeugt oder das Erscheinen von Bildlaufleisten bewirkt, wenn die Inhalte größer sind als die Zeile bzw. Spalte.

Beim Grid-Layout handelt es sich um ein **zweidimensionales** Design, welches in Zeilen **und** Spalten umgesetzt wird. Das erste Element in der zweiten Zeile eines solchen Layouts ist also nicht aufgrund eines Umbruchs dort zu finden, sondern weil es bewusst an diese Stelle platziert wird.

Bei der Verwendung des Grid-Layouts ist zu beachten, dass dieses nicht durch die Koordinaten der im Raster entstehenden Felder, sondern durch die **Zeilen- und Spaltenlinien** dargestellt wird.

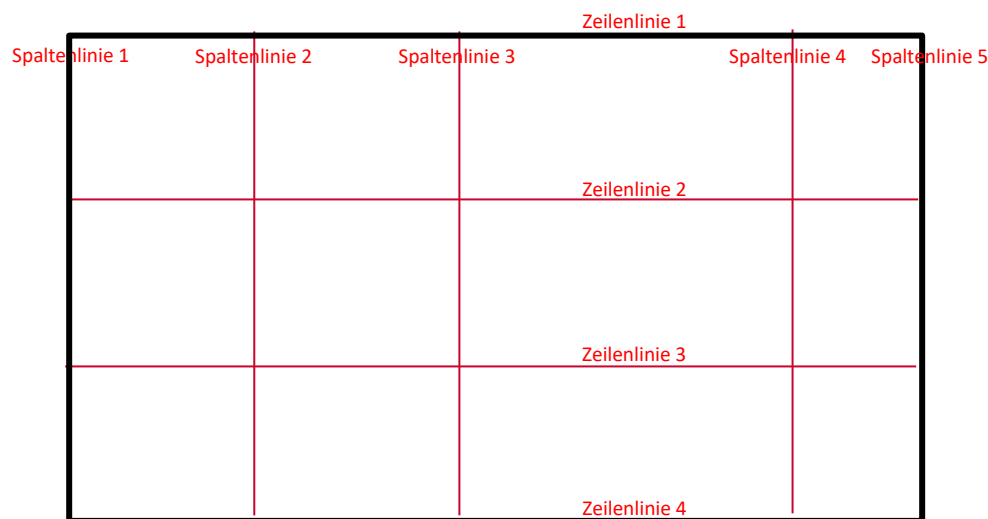


Abbildung 1: Definition der x- (Spaltenlinien) und y-Koordinaten (Zeilenlinien) in einem Grid-Layout

Die Zeilenlinien sind vom oberen Bereich des Rasters nach unten nummeriert. Die Spaltenlinien entsprechend von links nach rechts. Um ein Element in das Raster waagerecht in die erste und zweite Spalte zu platzieren, werden die Koordinaten also von Spaltenlinie 1 bis Spaltenlinie 3 festgelegt. Eine genaue Beschreibung ist weiter unten ab Seite 2 unter „grid-template-rows“ zu finden.

Ähnlich wie beim Flex-Layout sprechen wir beim Grid vom einem

- **Grid-Container** (dieser umfasst die Inhalte des Grids) und von
- **Grid-Item** (Elemente, auch komplexe Containerstrukturen, welche die eigentlichen Grid-Inhalte darstellen).

Ein Unterschied zum *Flex-Layout* ist allerdings, dass die Reihenfolge der zu platzierenden Elemente **nicht** von einer „order“-Eigenschaft, sondern von einer gezielten **Zuweisung in einem Koordinatensystem** abhängt.

Der Grid-Container

Der *Grid-Container* umfasst die im Raster anzuordnenden *Grid-Item*. Das Raster kann als „grid“ oder als „inline-grid“ deklariert werden. Die Deklaration „grid“ legt die gesamte Containerbreite für das Raster fest, was insbesondere bei der Festlegung automatischer Spaltenbreiten deutlich wird. Die Deklaration als „inline-grid“ macht dagegen die Breite vom Inhalt abhängig.

Es soll nun folgender HTML-Beispielcode verwendet werden, der durch weitere CSS-Codes ergänzt wird:

HTML-Code¹:

```
<body>
  <header>
    <h1>Dies ist eine Demoseite im Grid-Layout</h1>
  </header>
  <nav>
    <ul>
      <li><a href="#id1">Block 1</a></li>
...
      <li><a href="#id4">Block 4</a></li>
    </ul>
  </nav>
  <main>
    <article id="id1">
      <h2>Artikel 1</h2>
      <p>lorem ipsum ....</p>
    </article>
...
    <article id="id4">
...
      </article>
    </main>
  <footer>
...
  </footer>
</body>
```

Der HTML-Code zeigt eine einfache Struktur, wie sie in den ersten Web-Modulen (Module 3 bis 7) erklärt wurde. Dazu wird folgender CSS-Code angewendet (Farbformate sind hier nicht gezeigt. Die hellgrau unterlegten Code-Abschnitte werden im Verlauf dieses Moduls noch erläutert).

¹ Sich wiederholende Inhaltselemente wurden hier weggelassen und durch „...“ ersetzt.

```
body {
    display: grid;
    grid-template-rows: 80px auto 60px;
    grid-template-columns: 200px auto;
}
header {
    grid-row: 1;
    grid-column: 1 / 3;
}
nav {
    grid-row: 2;
    grid-column: 1;
}
main {
    grid-row: 2;
    grid-column: 2 / 3;
}
footer {
    grid-row: 3;
    grid-column: 1 / 3;
}
```

Das Ergebnis ist in Abbildung 2 zu sehen. Weil der `<body>` die volle Viewport-Breite einnimmt, ist das Raster auch über das gesamte Browserfenster gespannt. Die Regel „`display: grid;`“, mit der das `<body>`-Element als Grid-Container deklariert wird, bewirkt in Verbindung mit automatischen Spaltenbreiten, die vollständige Befüllung des Browsers in seiner Breite.

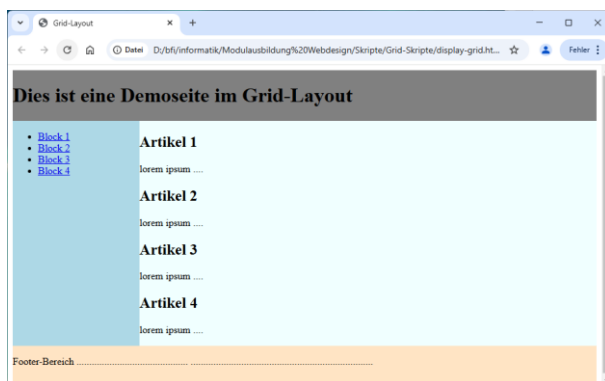


Abbildung 2: Das Element `<body>` wird als Grid-Container ("`display: grid;`") deklariert.

Der Code wird nun an einer Stelle verändert: Die Deklaration des Grid-Containers erfolgt jetzt mit der Regel „`display: inline-grid;`“. Alle übrigen Codezeilen bleiben unverändert.

```
body {
    display: inline-grid;
    grid-template-rows: 80px auto 60px;
    grid-template-columns: 200px auto;
}
```

Das Ergebnis zeigt nun Abbildung 3. Die Breite des Rasters – mindestens eine Spalte wurde in der Breite automatisch gesetzt – wird auf die maximale Inhaltsbreite gesetzt. In der Abbildung ist deutlich zu erkennen, dass am rechten Rand noch sehr viel Platz im Browserfenster verbleibt.

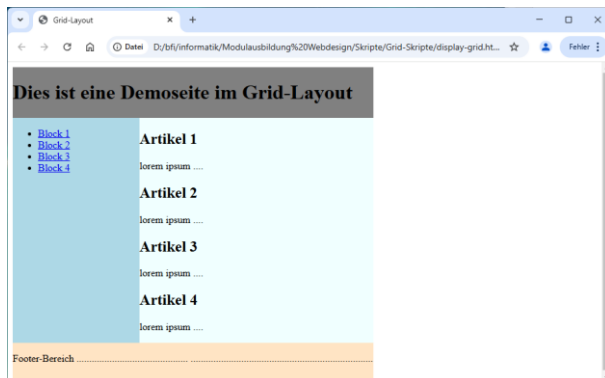


Abbildung 3: Auch hier wird das Element `<body>` als Grid-Container deklariert. Diesmal soll jedoch für die Eigenschaft „display“ der Wert „**inline-grid**“ anstelle von „**grid**“ verwendet werden. Es sind automatische Breiten-Deklarationen für die Spalten vorgenommen worden. Die Breiten dieser Spalten werden im Grid deswegen an die Inhalte ausgerichtet.

Struktur-Deklaration

In den folgenden Ausführungen wird ein vollkommen neues CSS-Format entwickelt!

Das Grid („Raster“) kann man sich wie eine Tabelle vorstellen, die aus Spalten und Zeilen besteht. Sowohl die Spalten als auch die Zeilen können verschiedene Breiten bzw. Höhen haben, welche in einer grundlegenden Deklaration festgelegt werden können. Es wird also eine Grid-Vorlage bzw. ein **Grid-Template** deklariert. Diese Deklaration erfolgt sowohl für die Zeilen (rows) als auch für die Spalten (columns).

Wichtig:

*Will man bei komplexen Webseiten nicht die Übersicht verlieren und riskieren, Elemente außerhalb des Rasters oder in der falschen Zeile wiederzufinden etc. dann ist es eigentlich schon eine **Selbstverständlichkeit**, ein vollständiges **Screenesign** (für jede Darstellungsform und jede Device-Art) zu entwerfen. Es sei hier auf das entsprechende Modul 10 in diesem Workshop verwiesen.*

Deklarationen können zeilenweise, spaltenweise oder für das gesamte Grid in einer gemeinsamen Deklaration und auf verschiedene Bereiche („Grid-Areas²“) bezogen programmiert werden.

grid-template-rows

Die CSS-Eigenschaft **grid-template-rows** legt die Höhe der **Zeilen** im Raster fest. Dafür gibt es verschiedene Möglichkeiten, die in der Regel auf jedem Browser zuverlässig funktionieren:

- Konkrete Angabe der Höhe mit Betrag und Maßeinheit (z.B. 70px)
- auto: Die Höhe der Zeile orientiert sich am Inhalt (mit dem größten Höhenbedarf).

Die folgende CSS-Regel, welche diese Anforderungen umsetzt, wird in das Format des Grid-Containers eingebunden.

```
grid-template-rows: 70px auto;
```

Abbildung 4 zeigt das Ergebnis, welches allerdings noch nicht an ein echtes „Raster“ erinnert. Lediglich die erste Zeile des Rasters wird – hier – mit einer Höhe von 70px formatiert. Die zweite Zeile wird durch das allgemeine Format „auto“ in der Höhe an den Inhalten der darin eingebundenen Elemente formatiert. Ebenso verhält es sich für folgende – nicht näher formatierte – Zeilen im Raster.

² Der Begriff „Grid-Area“ ist auch mit einer namentlichen Deklaration von Bereichen im Grid bekannt. Dies wird ab Seite 15 beschrieben.

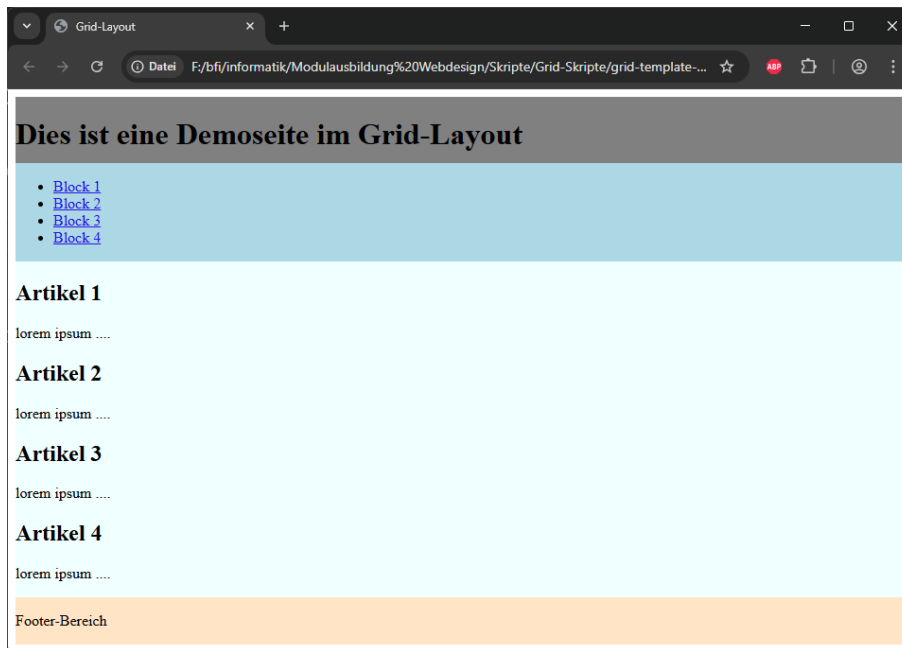


Abbildung 4: Es werden nur "Zeilen-Schablonen" definiert. Genau genommen greift die absolute Maßangabe für die erste Zeile. Alle weiteren Zeilen erhalten ihre Höhen automatisch von deren Inhalt.

grid-template-columns

Auffällig ist, dass alle Elemente die volle Breite des Viewports einnehmen. Hier wäre das Grid-Layout allerdings wenig sinnvoll. Es sollen auch Spalten in einer definierten Breite angelegt und zur Struktur der Seite eingesetzt werden.

In den Regelblock des Grid-Containers – hier: `<body>` – wird eine weitere CSS-Zeile eingefügt:

`grid-template-columns: auto auto;`

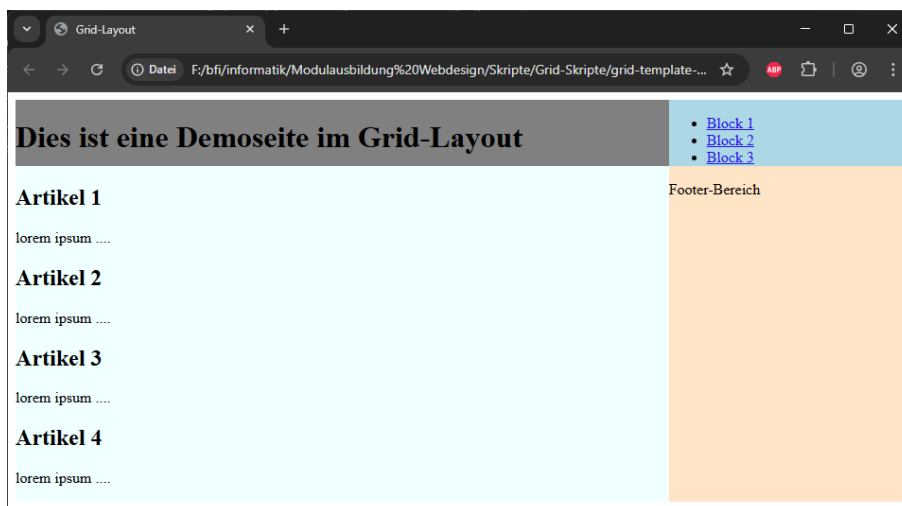


Abbildung 5: Nun ist das Raster mit zwei Spalten (jeweils automatische Breite) und zwei Zeilen (die erste Zeile mit der Höhe von 70px) formatiert.

Diese Regel definiert zwei Spalten im Raster, deren Breite automatisch zu wählen ist. Das bedeutet, dass sich die beiden Spalten – sofern die Inhalte nur sehr kurz sind – die verfügbare Breite im Viewport gleichmäßig teilen. In Abbildung 5 ist dies nicht der Fall. Der Text im grau unterlegten Element (hier: eine Überschrift `<h1>` im Element `<header>`) nimmt sich mehr Platz als die beiden anderen Elemente in der rechten Spalte. Neben dem Platz, den hier die Elemente benötigen, wird die überschüssige Breite auf die Spalten gleichmäßig aufgeteilt.

Es soll der Inhalt der <h1>-Überschrift im <header> durch einen längeren Text verändert werden. Die Seite reagiert direkt und passt die Spaltenbreite automatisch an. Die Breite der rechten Spalte wird dabei auf das unbedingt erforderliche Maß reduziert. Die rechte Spalte wird also direkt durch deren Inhalte begrenzt. Der Inhalt der ersten Zeile in der linken Spalte ist nun jedoch so groß, dass er nicht mehr in das Raster passt, ohne dass dafür die rechte Spalte noch weiter in der Breite reduziert werden müsste. Der Inhalt dieses Rasterfeldes wird also umbrochen.

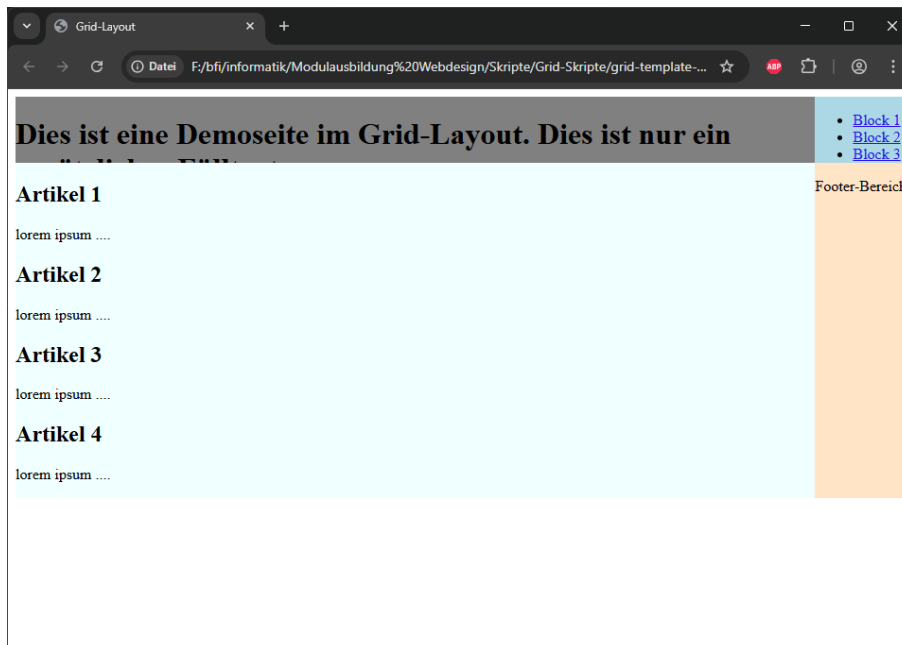


Abbildung 6: Der Text in der ersten Spalte (erste Zeile) ist länger geworden. Wegen der automatischen Spaltenbreite schiebt dieser Text die zweite Spalte soweit zusammen, dass deren Inhalt gerade noch hineinpasst.

Die bisherigen Ergebnisse können natürlich keinesfalls zufriedenstellend sein. Die Aufteilung einer Webseite sollte auf Grundlage in Breite und Höhe geplanter Elemente erfolgen und automatische Größeneinteilungen die Ausnahme sein. Grundlage für die Zuweisung der Abmessungen ist der Entwurf im Screendesign (Modul 10).

Es sollen in diesem Beispiel konkrete Forderungen umgesetzt werden:

- Der Header soll die volle Viewport-Breite und eine Höhe von 70px einnehmen.
- Das Menü soll in der Form von Schaltflächen erscheinen (vgl. Modul 1)
- Die linke Spalte soll eine Breite von 190px haben und das Menü enthalten.
- Der Hauptteil soll rechts neben dem Menü in der rechten Spalte erscheinen.
- Der Footer soll wieder die volle Breite des Viewports einnehmen und eine Höhe von 100px haben.

Die Zeilen- und Spaltenstruktur lässt sich mit dem folgenden CSS-Format darstellen. Dennoch wird das Ergebnis die Forderungen noch nicht erfüllen können.

```
body {
  display: grid;
  grid-template-rows: 70px auto 100px;
  grid-template-columns: 190px auto;
}
```

grid-template

Anstelle der getrennten Eigenschaften *grid-template-rows* (Zeilen-Deklaration) und *grid-template-columns* (Spalten-Deklaration) können diese Formate auch in einer einzigen CSS-Zeile festgelegt werden. Dazu wird die CSS-Eigenschaft „*grid-template*“ verwendet. Als Werte folgen nach dem Doppelpunkt zuerst die Zeilen-Deklarationen (jede Zeilenhöhe wird wie üblich mit einem Leerzeichen von der nächsten getrennt) und anschließend – getrennt durch ein Slash „/“ – die Werte für die Spaltenbreiten. Auch diese werden wieder durch Leerzeichen getrennt.

```
body {
    display: grid;
    grid-template: 70px auto 100px / 190px auto;
}
```

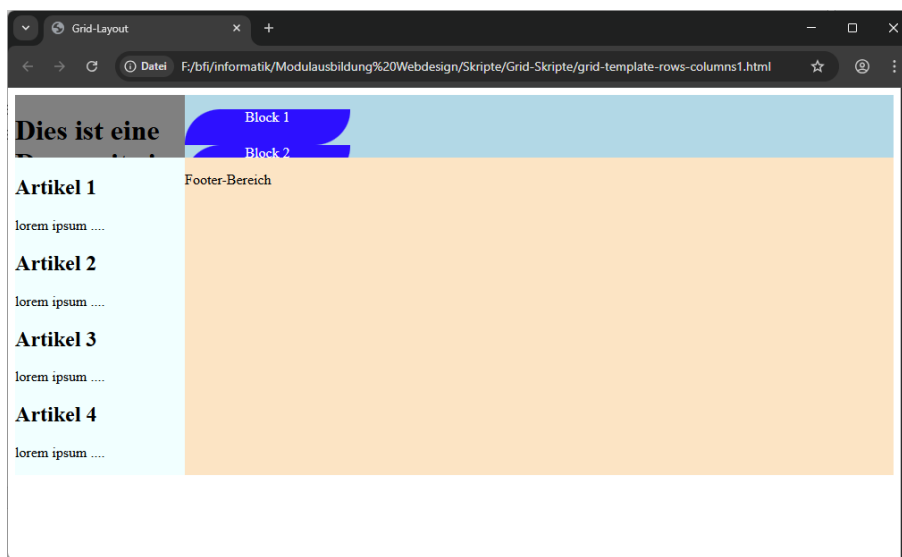


Abbildung 7: Ohjeh! Was ist denn hier passiert? - Das Menü sollte doch ganz woanders erscheinen.

Einordnung der Grid-Item in das Raster

Wie Abbildung 7 deutlich zeigt, reicht es in den meisten Fällen nicht aus, einfach das Raster aus Spalten und Zeilen sowie deren Abmessungen in den Templates zu deklarieren. Die einzelnen Elemente – die sogenannten Grid-Item – müssen gezielt im Raster angeordnet werden. Die dafür geeigneten Regeln werden direkt in die Formate der jeweiligen Elemente geschrieben. Zunächst sollen zwei CSS-Eigenschaften vorgestellt werden, mit deren Hilfe die Webseite die gewünschte Gestalt erhält:

- `grid-column`
- `grid-row`

grid-column

Das vorangegangene Beispiel soll im CSS-Code im Format für das Element `<header>` um eine Zeile ergänzt werden:

```
header {
    background-color: grey;
    grid-column: 1 / 3;
}
```

Erinnern wir uns an Abbildung 1: In diesem Bild wurden die Rasterlinien für die Spalten und Zeilen des Grid beschrieben. Spaltenlinie 1 ist demnach die linke Begrenzung des gesamten Rasters. Die Definition der „Koordinaten“ im Grid bezieht sich also nicht auf die Spalten selbst, sondern auf die sie begrenzenden Linien.

Im Beispiel wurde ein Raster mit zwei Spalten definiert. Zur Erinnerung: Im CSS-Format für den Body taucht die grün hervor gehobene CSS-Zeile auf:

```
body {
    display: grid;
    grid-template-rows: 70px auto 100px;
    grid-template-columns: 190px auto;
}
```

Das Raster wird also durch insgesamt drei (!) Spaltenlinien definiert:

1. linke Spaltenlinie
2. Grenzlinie zwischen der ersten und zweiten Spalte
3. rechte Begrenzungslinie des Rasters

Damit nun ein Element in seiner Breite das gesamte Raster überspannt, muss es also bei der ersten Begrenzungslinie beginnen und an der dritten Begrenzungslinie enden. So ergibt sich – obgleich es ja nur zwei Spalten in diesem Raster geben soll – die CSS-Regel für das Element <header>

grid-column: 1 / 3;



Abbildung 8: Es wurde nur für das Element <header> die Platzierung über beide Spalten des Grid festgelegt. Dessen Position ist nun wunschgemäß. Alle übrigen Elemente werden ohne zusätzliche Deklaration in der Reihe ihrer Programmierung im HTML-Code in das Raster eingefügt. Daher ist das Element <footer> noch nicht korrekt im Raster angeordnet.

Alternative Begrenzungsdefinition von rechts nach links

Anstatt die Begrenzungslinien von links nach rechts durchzuzählen, ist es auch möglich, die rechte Begrenzung des Grids durch eine negative Zahl festzulegen. So hat die folgende CSS-Regel das gleiche Ergebnis wie es in Abbildung 8 gezeigt wurde.

grid-column: 1 / -1;

Hinweis:

Der Wert „-1“ in der Deklaration der Zielbegrenzungslinie steht für die rechte Begrenzungslinie des gesamten Grid. Mit „-2“ wird demnach die zweite Linie von rechts gesehen bezeichnet.

„span“ - Alternative Definition durch Anzahl der nachfolgenden Linien

Den Begriff „span“ könnte man mit „überspannen“ übersetzen. Um ebenfalls das Ergebnis wie in Abbildung 8 zu erhalten, kann die CSS-Zeile auch folgendermaßen geschrieben werden:

```
grid-column: 1 / span 2;
```

Achtung! – Der Wert hinter span ist NICHT die Nummer der Begrenzungslinie, sondern gibt an, wie viele Begrenzungslinien – ausgehend von der Startlinie – zu überspringen sind. In diesem Fall darf also die Anzahl der Spalten genannt werden, welche das Element belegen soll.

Die Deklaration mit span ist also vorteilhaft, wenn ein mehrspaltiges Element in einem größeren Raster zu platzieren und die Zahl der zu belegenden Spalten bekannt ist. Es ist dann lediglich vor dem Slash die Begrenzungslinie festzulegen, an der das Element beginnen soll.

CSS-Eigenschaften für Start und Ende des Spaltenbereiches

Während die CSS-Regel „grid-column: 1 / 3;“ sowohl die Begrenzungslinie für den Start als auch für das Ende des Elements im Raster kombiniert beschreibt, können beide Parameter auch getrennt deklariert werden. So kann das <header>-Element auch mit den folgenden Regeln formatiert werden, um das Ergebnis in Abbildung 8 zu erhalten:

```
grid-column-start: 1;
```

```
grid-column-end: 3;
```

Arbeitsauftrag:

Lesen Sie noch einmal den Anforderungskatalog auf Seite 9. Ergänzen Sie die fehlenden Spaltendeklarationen für die Elemente <nav>, <main> und <footer>!

grid-row

In gleicher Weise wie die Spalten können auch die Zeilen als „Koordinaten“ betrachtet werden. Der obere Rand des Grids, also nicht die Mitte der ersten Inhaltszeile, ist dann die erste Zeile im Grid. Dementsprechend gibt es also eine Zeilenlinie mehr als Inhaltszeilen existieren.



Abbildung 9: Nicht schön, aber ein Klassiker: Header oben (ganze Seitenbreite), Footer unten (ebenso über die volle Seite), Menü links und Hauptteil rechts davon - Diese Anordnung ist in der Reihenfolge der im HTML-Code programmierten Elemente im Grid entstanden.

Im vorangegangenen Abschnitt wurde beschrieben, wie die Spalten mit den Elementen belegt werden können. Der Header und der Footer der gezeigten Beispiele sollen die volle Seitenbreite einnehmen und sind deswegen jeweils über zwei Spalten gelegt worden. Nun soll ein anderes Modell zur Seitenansicht gezeigt werden: Das Menü nimmt die gesamte linke Spalte ein. Dazu müssen nicht nur die Spalten-Koordinaten des Headers und des Footers angepasst werden, sondern es muss auch der Navigations-Container über die volle Seitenhöhe platziert werden.

Im folgenden CSS-Code – dieser zeigt die Formate der Grid-Item, der Grid-Container bleibt wie gehabt – wird die Start-Koordinate der Spalten beim Header und beim Footer auf „2“ geändert. Die erste Spalte soll ausschließlich der Navigation vorbehalten sein. Die Zeilenkoordinaten bleiben unverändert.

Im Navigationscontainer bleiben dagegen die Spaltenkoordinaten unverändert. Das Menü soll nach wie vor in der linken (ersten) Spalte erscheinen, jedoch die gesamte Höhe der Seite einnehmen. Es muss also die Eigenschaft `grid-row` für das Element `<nav>` angepasst werden (grüne, fett hervorgehobene Zeile im folgenden CSS-Code).

```
header {  
    grid-column: 2 / 3;  
    grid-row: 1;  
}  
nav {  
    grid-column: 1 / 2;  
    grid-row: 1 / 4;  
}  
main {  
    grid-column: 2 / 3;  
    grid-row: 2;  
}  
footer {  
    grid-column: 2 / 3;  
    grid-row: 3;  
}
```

Wie auch bei der Deklaration der Spaltenkoordinaten gibt es auch für die Zeilenkoordinaten alternative Schreibweisen. So kann anstelle der Zeile

```
grid-row: 1 / 4;
```

auch geschrieben werden:

```
grid-row: 1 / span 3;
```

oder

```
grid-row: 1 / -1;
```

„-1“ kann hier gesetzt werden, weil der Navigations-Container die komplette erste Spalte über alle Zeilen hinweg belegen soll. Nicht zuletzt lassen sich die Koordinaten für die Anfangs- und die Schlusszeile des Raster.Bereiches auch in einzelnen Regeln deklarieren:

```
grid-row-start: 1;
```

```
grid-row-end: 4;
```



Abbildung 10: Mit der CSS-Eigenschaft `grid-row` kann der Bereich im Grid über Zeilen hinweg deklariert werden. Im Beispiel: `grid-row: 1 / 4` (dreizeiliger Inhaltsbereich).

Abstände im Raster: `gap`, `row-gap` und `column-gap`

Wie u.a. in Abbildung 10 zu sehen ist, können mithilfe der CSS-grid-Technologie die HTML-Elemente innerhalb eines grid-Containers nahezu beliebig angeordnet werden. Dies ist unabhängig von der Reihenfolge wie sie im HTML-Dokument programmiert werden (korrekte Container-Struktur vorausgesetzt). Das Bild zeigt jedoch auch, dass die gezeigten Elemente direkt und ohne Abstand aneinandergesetzt wurden. Das ist nicht immer gewünscht.

Abstände zwischen(!) den Grid-Item können mit den folgenden CSS-Eigenschaften festgelegt werden:

- `gap` (Abstände von Zeilen und Spalten)
- `row-gap` (Abstand zwischen den Zeilen) und
- `column-gap` (Abstand zwischen den Spalten).

Hinweis:

Die Formate für die Zeilen- und Spaltenabstände werden im Regelblock des Grid-Containers deklariert.

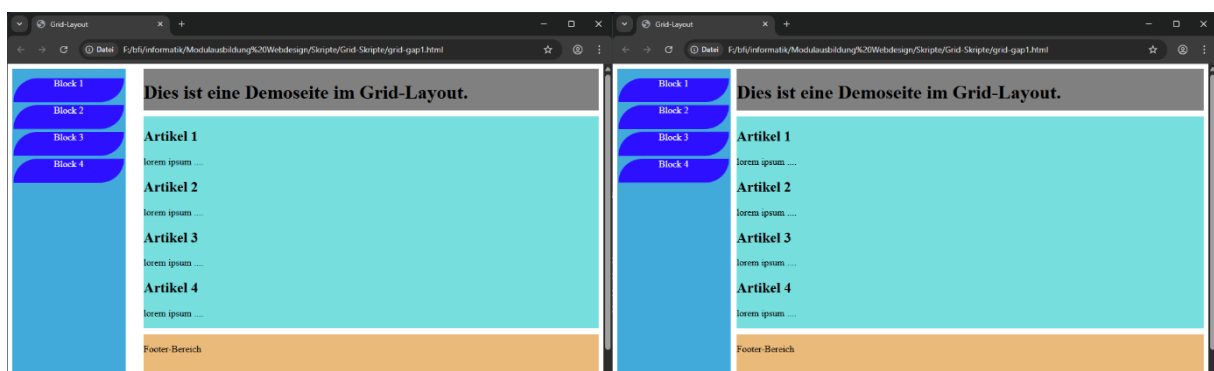


Abbildung 11: Beinahe das gleiche Bild wie zuvor, jedoch sind hier zwischen den Zeilen und Spalten Abstände gesetzt worden. Das Bild links sieht einen Zeilenabstand von 10px und einen Spaltenabstand von 20px vor. Im Bild rechts sind Spalten- und Zeilenabstände von jeweils 10px gesetzt worden. Beide Formate wurden mit der Eigenschaft `„gap“` im Grid-Container deklariert.

Die Eigenschaft `„gap“` formatiert gleichzeitig Zeilen- und Spaltenabstände. Dabei können in nur einer einzigen CSS-Regel die Abstände beider Dimensionen festgelegt werden. Der folgende CSS-Code verwendet für `„gap“` nur einen Wert. Das bedeutet, dass Zeilen- und Spaltenabstände mit dem gleichen Wert – hier: 10px – festgelegt werden.

```
body {
  display: grid;
  grid-template-rows: 70px auto 100px;
  grid-template-columns: 190px auto;
  gap: 10px;
}
```

Sollen den Zeilen- und Spaltenabständen verschiedene Werte zugewiesen werden, so ist dies ebenfalls mit der CSS-Eigenschaft „gap“ möglich. Hier werden zwei Parameter – zuerst der Zeilenabstand, dann durch ein Leerzeichen getrennt, der Spaltenabstand – festgelegt:

```
gap: 10px 30px;
```

Abbildung 11 Zeigt das Ergebnis.

Für die Formatierung von Zeilen- und Spaltenabstände gibt es auch individuelle Eigenschaften.

Abbildung 12 zeigt die Ergebnisse: Links wurde nur ein Spaltenabstand von 10px (*column-gap: 10px;*) und rechts nur ein Zeilenabstand von ebenfalls 10px (*row-gap: 10px;*) festgelegt.

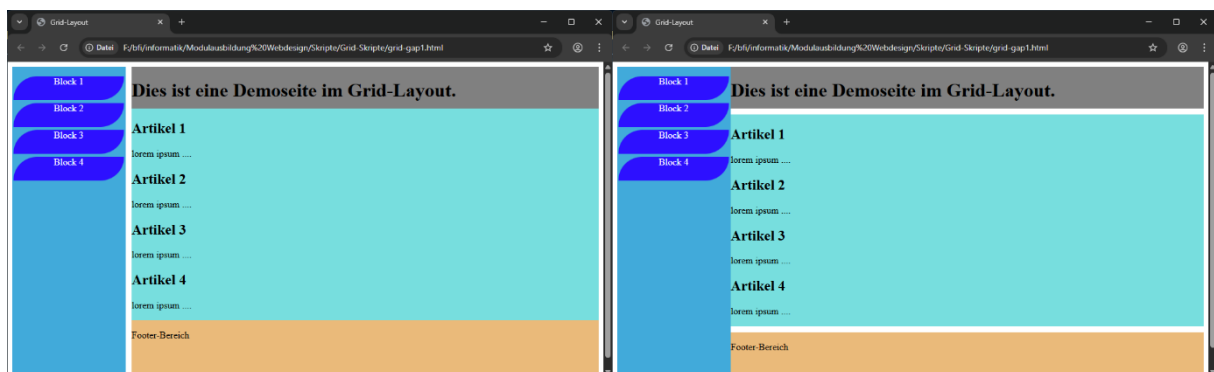


Abbildung 12: In den beiden Bildern wurde links nur ein Spaltenabstand und rechts ein Zeilenabstand deklariert.

grid-area

Man kann Elementen einen Namen für die Verwendung innerhalb des Rasters geben. Damit kann man sich die Orientierung möglicherweise erleichtern, wenn die Elemente keinen semantischen Namensbezug haben. Grundsätzlich lassen sich aber auch Grid-Areas gezielt mit Zeilen- und Spalten-Koordinaten im Raster anordnen.

Das in diesem Modul verwendete Beispiel kann auch mithilfe von Grid-Areas umgesetzt werden. Dabei sollen zunächst einmal die Höhen und Breiten der Zeilen und Spalten vernachlässigt werden. Der Grid-Container (hier: das Element <body> wird wie folgt definiert:

```
body {
  display: grid;
  grid-template-areas: "kopf kopf" "menue hauptteil" "fuss fuss";
}
```

grid-template-area

Auch hier handelt es sich wieder um ein Raster, bestehend aus drei Zeilen und zwei Spalten. Den jeweiligen Grid-Item wird in weiterer Folge jeweils ein – möglichst aussagekräftiger – Name gegeben. Dier kann weitgehend frei gewählt werden.

Die Deklaration „*grid-template-areas*“ gibt nun vor, wie die Felder in den Zeilen und Spalten zu belegen sind. Hier werden die beiden Spalten der ersten Zeile mit einer Grid-Area mit dem Namen „kopf“ belegt. Die Deklaration der Zeile wird in Hochkommata eingefasst. Es folgt die Deklaration der zweiten Spalte, welche hier mit den Grid-Areas „menue“ und „hauptteil“ belegt wird und schließlich die dritte Zeile, in der beide Spalten jeweils die Grid-Area „fuss“ beinhalten.

Die Vergabe der Namen der Grid-Areas findet wiederum in den Formaten der jeweiligen Grid-Item statt. Nachfolgend ist als Beispiel der CSS-Code für das Element <header> gezeigt, welches die Grid-Area mit dem Namen „kopf“ sein soll. Alle weiteren Grid-Item werden entsprechend formatiert.

```
header {
    grid-area: kopf;
}
```



Abbildung 13: Lediglich das *grid-area-template* und die Namen der zugehörigen *grid-areas* wurden festgelegt. Damit erscheinen die Elemente in automatisch nach ihren Inhalten in der Größe bemessenen Zeilen und Spalten. Es geht aber noch genauer!

Das Ergebnis in Abbildung 13 kann noch nicht professionellen Maßstäben entsprechen! Die Breiten und Höhen der Zeilen und Spalten im Raster müssen genauer beschrieben werden. Dazu werden die bereits erläuterten CSS-Eigenschaften „*grid-template-rows*“ und „*grid-template-columns*“ verwendet.

```
body {
    display: grid;
    grid-template-rows: 70px auto 100px;
    grid-template-columns: 190px auto;
    grid-template-areas: "kopf kopf" "menue hauptteil" "fuss fuss";
}
```



Abbildung 14: Das sieht besser aus: Auch Zeilen und Spalten wurden maßgenau definiert.

grid-area ohne Namen

Die Deklaration der „*grid-template-area*“ hat durchaus einen Vorteil, der sich in komplexeren Seiten positiv bei der Arbeit bemerkbar machen kann: Alle wesentlichen Formate des Grids – von der Deklaration der Zeilen- und Spaltenbreite bis zur Belegung der einzelnen Zellen des Rasters – werden in den Formaten des Grid-Containers festgelegt.

Es ist aber auch möglich, in den Formaten der Grid-Item die Eigenschaft „*grid-area*“ zur Deklaration des zu belegenden Bereiches zu verwenden. Im Grunde genommen entspricht dies aber der bereits beschriebenen Praxis mit „*grid-row*“ und „*grid-column*“.

Ausrichtung der Elemente

In diesem Modul wurde eine Technologie beschrieben, mit denen die Inhalte eines (Grid)-Containers strukturiert und diesen die gewünschten Positionen zugewiesen werden können. Dabei kann es sich um einzelne Bereiche innerhalb der Seite aber auch um die gesamte Webseite selbst handeln.

Offen ist jedoch die Frage geblieben, wie die einzelnen Inhalte der Rasterbereiche sowohl horizontal als auch vertikal ausgerichtet werden können. Hier kommen bereits im Modul 14 (Flex) vorgestellte CSS-Eigenschaften zum Einsatz:

- justify-content
- justify-items
- align-content
- align-items

Diese CSS-Eigenschaften dürfen gerne in eigenen Experimenten untersucht werden.

Übungsaufgaben

Aufgabe 15.1: Welche Bedeutung hat die Eigenschaft "order" in einem Grid-Layout?

Aufgabe 15.2: Was passiert, wenn Sie zwar den Grid-Container (inkl. Zeilen- und Spalten-Templates) deklarieren, jedoch in den Elementen, welche der Container umfasst, keine weiteren CSS-Grid-Formate festlegen?

Aufgabe 15.3: Sie haben ein dreispaltiges Grid deklariert: Was passiert, wenn Sie einem der Grid-Item die folgende CSS-Regel zuweisen: `grid-column: 1 / 5`?

Aufgabe 15.4: Programmieren Sie ein fiktives Menü, in dem anstelle von Schaltflächen kreisrunde „Kacheln“ mit dem Radius von 50px angeboten werden. Die (kreisförmigen) Kacheln sollen in einem Raster von 4 x 4 Flächen erscheinen. Funktionierende Links sind nicht erforderlich. Grafiken für die Hintergründe können frei gewählt werden.

Aufgabe 15.5: Programmieren Sie eine Beispiel-Webseite mit folgender Struktur:

- Ein Grid mit drei Zeilen und drei Spalten (Maße wie folgt)
- Header mit einer Höhe von 75px über die volle Seitenbreite
- Navigation mit einer Breite von 200px in der zweiten Zeile und ersten Spalte des Rasters
- Hauptteil mit automatischer Breite in der Mitte der Seite (nur zweite Zeile)
- Ein „Aside“-Element in der rechten Spalte (nur zweite Zeile), 150px Breite
- Footer mit einer Höhe von 50px über die volle Seitenbreite
- Geben Sie den Containern unterschiedliche Hintergrundfarben und befüllen Sie diese mit Fülltexten (z.B.: lorem ipsum)

Aufgabe 15.6: Erweitern Sie die Webseite aus Aufgabe 15.5 zu einer responsiven Seite, welche drei verschiedene Darstellungen bietet (Hinweis: Erinnern Sie sich an Modul 13 – Media-Queries):

- Viewport-Breite bis 700px: einspaltige Darstellung (Alle Elemente erstrecken sich über die volle Bildschirmbreite). Ordnen Sie die Elemente in folgender Reihenfolge an: Header, Navigation, Hauptteil, Aside-Container und Footer
- Viewport-Breite von 701px bis 1200px; zweispaltige Darstellung – Der Header wird über die volle Bildschirmbreite in die erste Zeile gesetzt, die Navigation in der ursprünglichen Spaltenbreite in die linke Spalte der zweiten Zeile und der Hauptteil rechts daneben mit automatischer Breite. Der Aside-Container belegt die dritte, der Footer die vierte Zeile jeweils über die volle Bildschirmbreite.
- Ab einer Viewport-Breite von 1201px soll die Darstellung wieder der Aufgabe 15.5 entsprechen.
- Überlegen Sie: Welche Grid-Formate müssen in welcher Form verändert werden?